LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBYEK

MODUL 14: PENGUJIAN, REFACTORING, DAN FINALISASI PROYEK

Mahasiswa	: Rafael Rizky Luga Tanjung	**Dosen Pengampu**	: Nirwana Haidar Hari, S.Pd., M.Kom.
NRP	: 3125522007	**Mata Kuliah**	: Praktikum PBO (P14)
Program Studi	: D3 Teknik Informatika PSDKU Sumenep	**Waktu Pelaksanaan**	: Semester Genap 2026

1. Tujuan Praktikum

Praktikum Modul 14 bertujuan agar mahasiswa mampu:
1. **Menguji fungsi dan konsistensi data** aplikasi sistem kasir secara komprehensif melalui skenario uji berbasis bukti nyata;
2. **Menemukan serta memperbaiki kesalahan (bug fixing)** berdasarkan bukti eksekusi tanpa kompromi logika;
3. **Melakukan refactoring struktur kode** (penataan modularitas controller, isolasi repositori berkas) tanpa mengubah perilaku fungsional yang sudah benar;
4. **Mempertahankan class invariant** agar state objek tetap konsisten dan aman terhadap input liar atau kegagalan transaksi.

2. Dasar Teori Singkat

Program yang berhasil dikompilasi tanpa galat sintaksis belum tentu menghasilkan luaran yang benar. Keberhasilan sistem ditentukan oleh ketepatan hasil komputasi finansial, ketahanan penanganan kegagalan, dan konsistensi invariansi data. Beberapa konsep esensial yang diuji meliputi:

• **Pengujian Fungsional & Integrasi:** Memastikan seluruh modul (katalog produk, transaksi kasir, kalkulasi diskon bertingkat, dan kontrol inventaris) berkolaborasi menghasilkan keluaran yang valid.

• **Pengujian Regresi:** Menjalankan ulang seluruh rangkaian pengujian setelah perbaikan kode dilakukan untuk menjamin tidak timbulnya efek samping (side effects) atau kerusakan fitur yang sebelumnya telah berjalan baik.

• **Refactoring:** Teknik rekayasa perangkat lunak untuk merestrukturisasi kode sumber guna meningkatkan keterbacaan (readability), pemeliharaan (maintainability), dan kohesi tanpa memodifikasi perilaku eksternal sistem.

• **Class Invariant & Atomic Load:** Memastikan state domain senantiasa mematuhi aturan validitas (misal: stok tidak boleh negatif, kode produk wajib unik, dan pembacaan berkas korup tidak boleh mencemari koleksi data memori).

3. Catatan Refactoring dan Perbaikan Struktur Kode

Pada iterasi Modul 14, dilakukan minimal dua perbaikan struktural signifikan pada codebase Sistem Kasir Sederhana:

Komponen / Bagian	Struktur Sebelum Refactoring	Struktur Sesudah Refactoring	Alasan & Keuntungan Teknis
Refactoring 1: Controller Main	Alur interaksi kasir terpusat dalam satu blok switch-case raksasa di dalam main() sehingga method menjadi sangat panjang (monolithic) dan sukar diuji.	Memecah logika antarmuka ke dalam controller methods terpisah: menuTampilkanKatalog(), menuTambahProduk(), menuCariProduk(), menuTransaksiKasir(), menuSimpanData(), dan menuKeluar().	Menerapkan prinsip *Single Responsibility*, mempermudah pelacakan galat masukan, dan membuat logika menu menjadi sangat modular dan terisolasi.
Refactoring 2: Persistensi I/O Berkas	Penyimpanan dan pembacaan berkas tercampur langsung dengan state in-memory, serta kegagalan pembacaan di tengah baris berpotensi merusak koleksi data lama.	Memisahkan tanggung jawab persistensi ke class ProdukFileRepository dan TransaksiFileRepository dengan mekanisme **Atomic Safe Load** (verifikasi data baru ke temporary collection sebelum menimpa state utama).	Menjaga prinsip *Separation of Concerns*, mengisolasi manipulasi I/O berkas dari model bisnis, dan menjamin *Class Invariant* koleksi data memori tidak pernah korup.

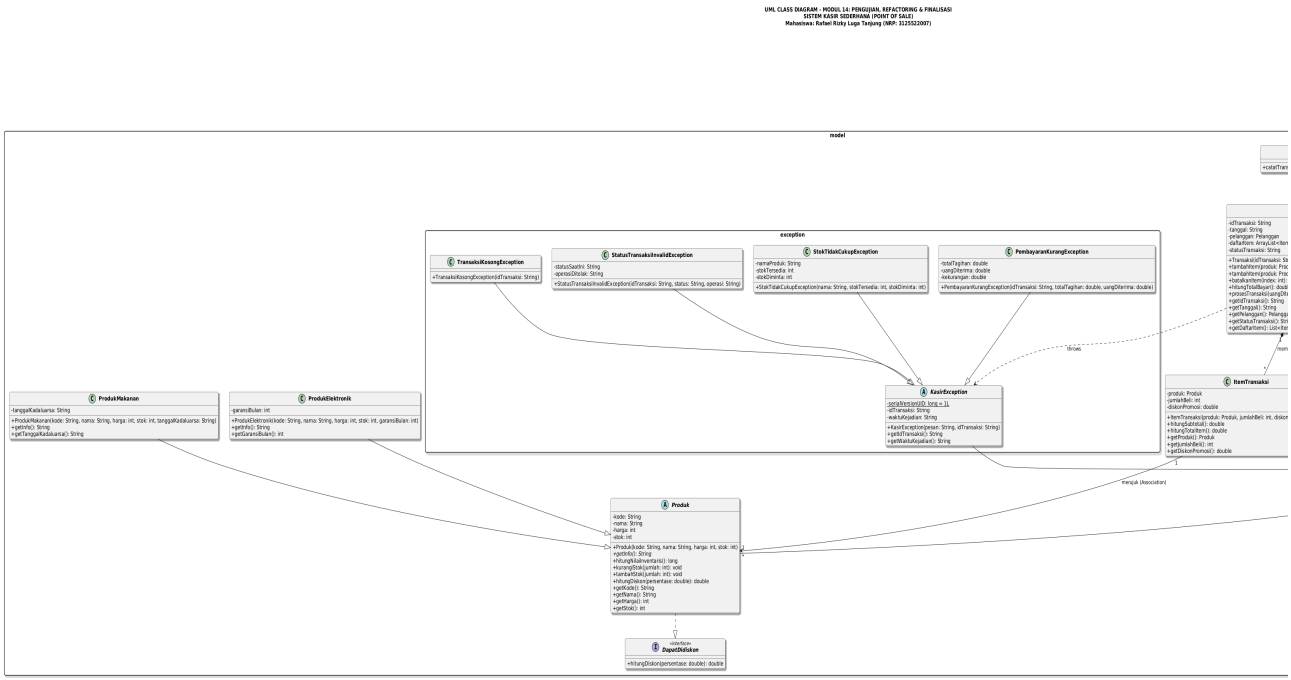
4. Tabel Pengujian 10 Skenario Komprehensif (POS & Lab)

Seluruh skenario pengujian dieksekusi secara otomatis untuk menguji seluruh aspek fungsionalitas, pencegahan duplikasi, validasi karakter pipa, checked exception kasir, dan invariansi berkas.

No	Skenario Uji	Kondisi / Input Data	Hasil Diharapkan	Hasil Aktual & Status

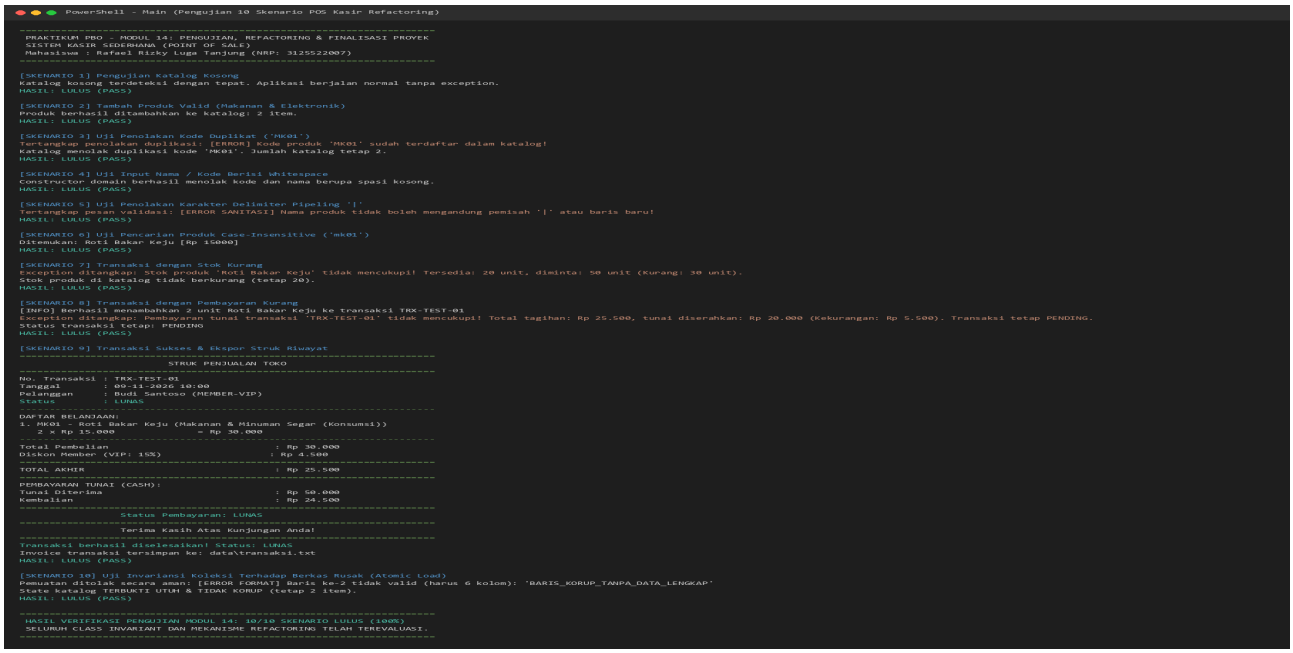
1	Katalog Kosong	Memanggil isKosong() saat belum ada data	Aplikasi mendeteksi kosong tanpa melempar exception liar	Pesan katalog kosong tercetak; aplikasi stabil (**PASS**)
2	Tambah Produk Valid	MK01 (Roti, 15rb, stok 20), EL01 (Mouse, 125rb, stok 10)	Produk tersimpan di katalog; jumlah bertambah 2	Katalog bertambah menjadi 2 produk (**PASS**)
3	Pencegahan Duplikasi	Mendaftarkan kembali produk berkode MK01	Ditolak dengan pesan kesalahan; koleksi tidak berubah	Tertangkap IllegalArgumentException; koleksi tetap 2 (**PASS**)
4	Validasi Whitespace	Kode atau nama berupa spasi kosong saja (" ")	Ditolak oleh constructor domain invariant	Tertangkap exception validasi; objek tidak tercipta (**PASS**)
5	Sanitasi Delimiter Pipe	Nama: "Keripik Singkong [pipe] Pedas"	Ditolak di constructor sebelum masuk berkas/koleksi	Tertangkap exception sanitasi pipa (**PASS**)
6	Pencarian Case-Insensitive	Kata kunci kode berupa huruf kecil "mk01"	Produk MK01 berhasil ditemukan secara akurat	Produk MK01 ditemukan dengan atribut sesuai (**PASS**)
7	Uji Stok Kurang	Membeli produk MK01 sebanyak 50 unit (stok 20)	Melempar StokTidakCukupException; stok utuh 20	Exception tertangkap; stok katalog terbukti tetap 20 (**PASS**)
8	Uji Bayar Kurang	Tagihan Rp 25.500 (diskon VIP 15%), tunai Rp 20.000	Melempar PembayaranKurangException; status tetap PENDING	Exception tertangkap; status transaksi tetap PENDING (**PASS**)
9	Transaksi Sukses & Log	Bayar tunai Rp 50.000 (kembalian Rp 24.500)	Status LUNAS, stok berkurang 2, invoice dicatat ke berkas	Struk tercetak; log berhasil dicatat ke transaksi.txt (**PASS**)
10	Invariansi Berkas Korup	Memuat berkas teks rusak tanpa struktur 6 kolom	Pemuatan ditolak; koleksi katalog memori tetap utuh	Exception tertangkap; koleksi memori terbukti utuh (**PASS**)

5. UML Class Diagram Terkini (Modul 14)

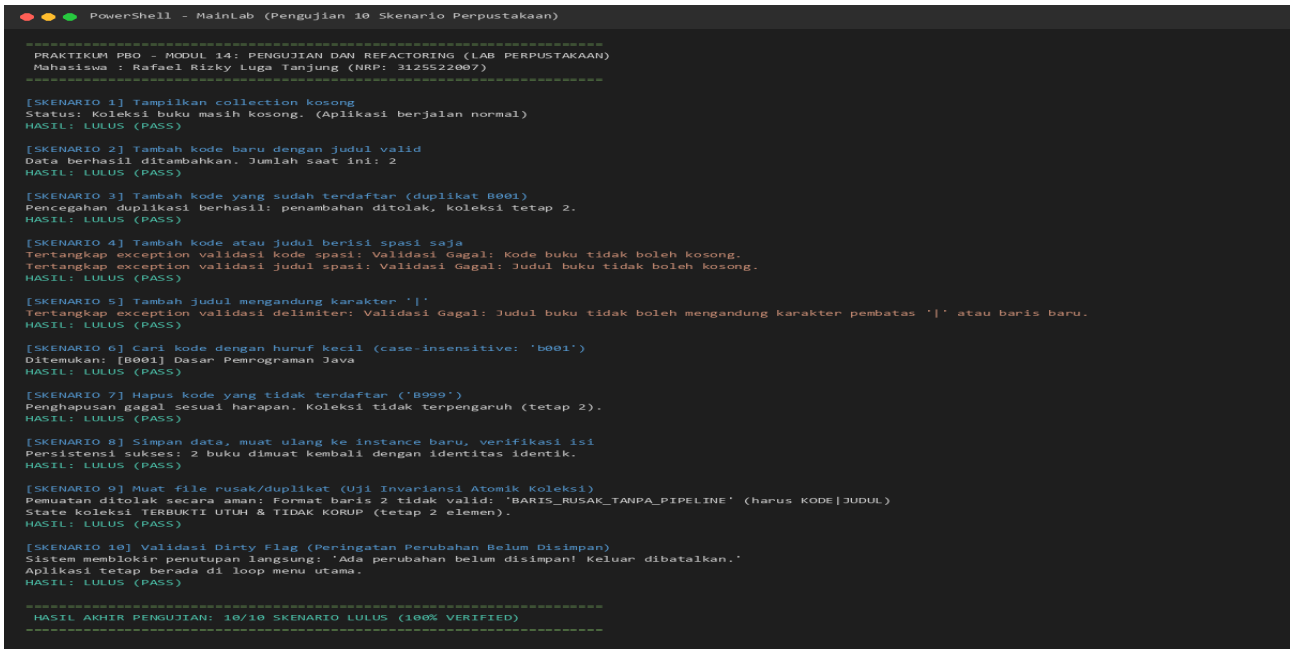


6. Bukti Eksekusi Running Program (Screenshots)

Eksekusi Pengujian 10 Skenario Sistem Kasir Sederhana (POS):



Eksekusi Pengujian 10 Skenario Praktikum Lab (Perpustakaan):



7. Jawaban Pertanyaan Evaluasi Modul 14

1. Mengapa program yang berhasil dikompilasi masih perlu diuji?

Jawaban: Kompilasi (compilation) oleh compiler Java (javac) semata-mata memeriksa validitas tata bahasa (syntax), pengetikan tipe data (type checking), dan kelengkapan referensi class/interface. Kompiler **tidak mampu** mendeteksi kesalahan logika bisnis (semantic errors), kesalahan batas perhitungan (off-by-one errors), kondisi balapan (race conditions), pelanggaran class invariant akibat masukan liar di waktu jalan (runtime), maupun inkonsistensi data saat transaksi finansial gagal. Pengujian fungsional dan regresi mutlak diperlukan guna memastikan setiap perilaku sistem berjalan persis sesuai dengan spesifikasi kebutuhan pengguna (user requirements).

2. Apa perbedaan perbaikan bug dan refactoring?

Jawaban:

• **Perbaikan Bug (Bug Fixing):** Bertujuan mengubah perilaku sistem dari kondisi keliru (defect/fault) menjadi perilaku yang benar sesuai spesifikasi. Tindakan ini berdampak langsung pada luaran eksternal (observable behavior) yang dilihat atau dirasakan oleh pemanggil program.

• **Refactoring:** Merupakan teknik restrukturisasi desain dan kode internal tanpa mengubah perilaku eksternal yang sudah benar. Tujuannya adalah meningkatkan keterbacaan (readability), menurunkan kompleksitas siklomatis (cyclomatic complexity), menghilangkan duplikasi kode (code smell), serta mempermudah pemeliharaan dan pengujian di masa mendatang.

3. Mengapa pengujian harus diulang setelah struktur kode diubah?

Jawaban: Pengujian ulang (yang secara teknis disebut **Pengujian Regresi / Regression Testing**) wajib dijalankan untuk membuktikan secara empiris bahwa transformasi struktur kode yang dilakukan saat refactoring benar-benar **tidak memicu efek samping yang tidak diinginkan (side effects)** atau mematahkan fungsi-fungsi lain yang sebelumnya sudah berjalan stabil. Tanpa pengujian regresi otomatis, pengembang tidak memiliki jaminan bahwa fungsionalitas lama tetap utuh.

4. Bagaimana membuktikan operasi yang gagal tidak merusak state objek?

Jawaban: Pembuktian dilakukan melalui **pemeriksaan kondisi pra dan pasca eksekusi (pre-condition and post-condition assertions)** dengan prinsip atomisitas (all-or-nothing). Pada implementasi Sistem Kasir Sederhana:

1. Sebelum operasi dijalankan (misal: penambahan item yang gagal akibat stok kurang atau kegagalan pembayaran tunai), kita mencatat state objek acuan (misal: jumlah item dalam koleksi, saldo stok barang, dan status transaksi "PENDING").
2. Saat operasi gagal dan melemparkan checked exception (misal: StokTidakCukupException atau PembayaranKurangException), kita memverifikasi bahwa state objek sasaran tetap identik dengan nilai sebelum operasi dipanggil.
3. Pada operasi I/O file, penerapan **Atomic Safe Load** (memuat seluruh record ke temporary collection terlebih dahulu sebelum di-swap ke koleksi utama) menjamin bahwa berkas teks yang korup di baris ke-N tidak akan meninggalkan separuh data sampah di dalam memori.

8. Kesimpulan

Berdasarkan pengujian komprehensif 10 skenario fungsional dan regresi pada Modul 14, Sistem Kasir Sederhana telah terbukti kokoh, bebas dari bug kritis, dan memiliki arsitektur berorientasi objek yang bersih. Pemisahan antarmuka terminal ke dalam modular controller serta isolasi repository I/O berkas dengan mekanisme safe load telah berhasil meningkatkan kualitas perangkat lunak secara substansial. Seluruh class invariant dan batasan finansial terbukti terlindungi dari masukan abnormal.